

AGENTIC AI

Complete Learning & Interview Preparation Syllabus

LLMs • RAG • Tool Calling • Agents • Memory • Multi-Agent Systems
LangChain • LangGraph • MCP • Reliability • Security • Evaluation • Architecture

19 Core Sections	Critical + High Priority Focus	Theory + Hands-On Interview Ready
----------------------------	--	---

Designed as a structured roadmap for Agentic AI engineering, practical implementation, system design, and interview preparation.

Recommended Learning Order

LLM → Prompts → Embeddings → RAG → Tool Calling → Agents → State & Memory → Workflows → Multi-Agent → LangChain → LangGraph → Reliability → Security → MCP → Evaluation & Observability → Architecture → Hands-On → Interview Scenarios

Use the checkboxes to track progress. Critical sections should be prioritized before High-priority sections.

1. LLM Fundamentals

CRITICAL

- What is an LLM?
- How LLMs Work - High-Level Architecture
- Transformer Basics
- Attention / Self-Attention - Conceptual Understanding
- Parameters and Weights
- Pre-training
- Training vs Inference
- Next-Token Prediction
- Tokens
- Tokenization
- BPE / Tokenizer Basics
- Context
- Context Window
- Context vs Memory
- Prompts
- System / User / Assistant Messages
- Temperature
- Hallucination
- Structured Output
- Base Model vs Instruction-Tuned Model
- LLM vs SLM
- Reasoning Models
- Fine-Tuning - Conceptual Understanding
- Full LLM Request -> Response Flow

2. Prompt Engineering

CRITICAL

- What is Prompt Engineering?

- System Prompts
- User Prompts
- Instructions vs Context
- Zero-Shot Prompting
- Few-Shot Prompting
- Role Prompting
- Prompt Templates
- Structured Prompting
- Structured Outputs / JSON
- Prompt Constraints
- Context Management
- Common Prompt Failures
- Prompt Injection Basics

3. Embeddings + Vector Databases

CRITICAL

- What are Embeddings?
- How Text Becomes a Vector
- Embedding Models
- Vector Dimensions
- Semantic Similarity
- Cosine Similarity - Conceptual
- Vector Databases
- Vector DB vs Traditional DB
- Similarity Search
- Top-K Retrieval
- Metadata
- Metadata Filtering
- Semantic Memory + Vector DB Relationship

4. RAG - Retrieval-Augmented Generation

CRITICAL

- What is RAG?
- Why RAG is Needed
- RAG vs Fine-Tuning
- Knowledge Base

- Ingestion
- Document Loading
- Parsing
- Cleaning
- Chunking
- Chunk Size
- Chunk Overlap
- Embedding Generation
- Vector Storage
- Query Embedding
- Semantic Search
- Keyword Search
- Hybrid Search
- Retrieval
- Top-K
- Reranking
- Context Construction
- Grounding
- Citations / Source Attribution
- RAG Hallucinations
- Offline / Indexing RAG Flow
- Online / Query RAG Flow
- End-to-End RAG Architecture

5. Tool / Function Calling

CRITICAL

- What is Tool Calling?
- Why Agents Need Tools
- Tool Definitions
- Tool Schemas
- Tool Arguments
- Tool Selection
- Argument Validation
- Tool Execution
- Returning Tool Results to the LLM
- Sequential Tool Calls

- Parallel Tool Calls
- Tool Errors
- Retries
- Timeouts
- Tool Permissions
- Tool Authorization
- Tool Calling vs API Calling
- Complete Tool-Calling Flow

6. AI Agents

CRITICAL

- What is an AI Agent?
- Agent vs LLM
- Agent vs Chatbot
- Agent vs RAG
- Agent vs Deterministic Workflow
- Components of an Agent
- Agent Loop
- Goal / Task
- Planning
- Reasoning
- Action
- Observation
- ReAct
- Reason -> Act -> Observe
- Tool Selection
- Dynamic Decision-Making
- Termination Conditions
- Agent Execution Flow

7. State + Memory

CRITICAL

- What is State?
- Agent State
- Workflow State
- How State / Workflows are Defined

- State vs Memory
- Conversation / Short-Term Memory
- Long-Term / Persistent Memory
- Semantic Memory
- Episodic Memory
- Working Memory
- Session State
- User Memory
- Memory Storage
- Memory Retrieval
- Vector DB for Memory
- Checkpointing
- State Persistence
- State Recovery

8. Agentic Workflows + Orchestration

CRITICAL

- What is Agent Orchestration?
- Deterministic vs Agentic Workflows
- Sequential Workflows
- Parallel Workflows
- Conditional Workflows
- Branching
- Loops
- Routing
- Workflow State
- Dynamic Routing
- Human-in-the-Loop
- Approval Steps
- Workflow Termination
- Combining Deterministic Logic with LLM Decisions

9. Multi-Agent Systems

CRITICAL

- What is a Multi-Agent System?
- Single Agent vs Multi-Agent

- Supervisor / Worker Pattern
- Specialized Agents
- Router Pattern
- Agent Handoffs
- Shared State
- Shared Memory
- Parallel Agent Execution
- Agent Communication
- Coordination
- Aggregating Agent Results
- Multi-Agent Failure Handling
- When to Use Multi-Agent
- When NOT to Use Multi-Agent
- Multi-Agent Architecture Example

10. LangChain

CRITICAL

- What is LangChain?
- Why LangChain?
- LangChain Architecture
- Chat Models
- Messages
- Prompt Templates
- Structured Output
- Output Parsers
- Chains / Runnable Composition
- Tools
- Tool Calling
- Retrievers
- Document Loaders
- Text Splitters
- Embeddings
- Vector Stores
- Building RAG with LangChain
- Agents
- Agent Execution

- Memory / Persistence Concepts
- Error Handling
- Small End-to-End LangChain Application

11. LangGraph

CRITICAL

- What is LangGraph?
- Why LangGraph?
- LangChain vs LangGraph
- Graph-Based Agent Workflows
- StateGraph
- State
- Nodes
- Edges
- START
- END
- Conditional Edges
- Routing
- Loops / Cycles
- ToolNode
- Agent Node
- State Updates
- Reducers - Basics
- Checkpointing
- Persistence
- Threads / Sessions
- Human-in-the-Loop
- Interrupt / Resume Concepts
- Error Handling
- Retry Strategies
- Multi-Agent Workflows
- Supervisor Agent with LangGraph
- End-to-End LangGraph Agent

12. Reliability + Failure Handling

CRITICAL

- Why Agent Reliability is Difficult

- Max Steps

- Max Iterations

- Max Tool Calls

- Timeouts

- Retry

- Exponential Backoff

- Duplicate Tool-Call Detection

- Infinite Loop Prevention

- Repeated-Error Fingerprinting

- Idempotency

- Circuit Breaker

- Fallback

- Model Fallback

- Tool Fallback

- Graceful Degradation

- Human Escalation

- Partial Failure Handling

- Recovery Strategies

13. Security + Guardrails

CRITICAL

- Agentic AI Security Risks

- Prompt Injection

- Direct Prompt Injection

- Indirect Prompt Injection

- Jailbreaking - Conceptual

- Data Leakage

- Tool Abuse

- Least Privilege

- Tool Authorization

- Authentication vs Authorization

- Input Validation

- Output Validation

- Schema Validation

- Sensitive Data / PII Handling

- Secrets / API-Key Management
- User Confirmation for Sensitive Actions
- Allowlisting Tools / Actions
- Guardrails
- Human Approval for High-Risk Operations

14. MCP - Model Context Protocol

HIGH

- What is MCP?
- Why MCP Exists
- MCP Architecture
- MCP Host
- MCP Client
- MCP Server
- MCP Tools
- MCP Resources
- MCP Prompts
- How an LLM/Agent Interacts with MCP
- MCP vs Function Calling
- MCP vs REST API
- MCP Security Considerations
- Example MCP Architecture

15. Evaluation

HIGH

- Why Agent Evaluation is Difficult
- Task Success
- Response Quality
- Tool-Call Accuracy
- Tool Selection Accuracy
- Argument Accuracy
- RAG Retrieval Quality
- RAG Relevance
- Faithfulness
- Groundedness
- Hallucination Evaluation

- LLM-as-Judge
- Human Evaluation
- Test Datasets
- Regression Testing
- Agent Evaluation Scenarios

16. Observability

HIGH

- Why AI Observability is Needed
- Logs
- Traces
- Agent Traces
- Tool-Call Traces
- LLM Calls
- Prompt / Response Tracking
- Latency
- Token Usage
- Cost
- Error Rate
- Tool Success Rate
- RAG Retrieval Monitoring
- Debugging Agent Workflows

17. Agentic AI Architecture / System Design

CRITICAL

- End-to-End Agentic AI Architecture
- Client -> API -> Agent Flow
- API Gateway
- Agent / AI Orchestrator
- LLM Layer
- Tool Layer
- RAG Layer
- Vector Database
- Application Database
- State Store
- Memory Store

- Cache
- External APIs
- Authentication / Authorization
- Guardrail Layer
- Observability Layer
- Message Queues / Async Processing
- Human Approval Layer
- Scaling Agents
- Handling Concurrent Users
- Latency Optimization
- Cost Optimization
- Model Selection
- Failure Handling
- Designing Production-Ready Agents

18. Practical Hands-On

CRITICAL

- Basic LLM Call in Python
- Prompt -> LLM -> Response
- Structured JSON Output
- Simple Tool Calling
- Multiple Tools
- Sequential Tool Calls
- Parallel Tool Calls
- Simple RAG Application
- Document -> Chunk -> Embed -> Store
- Query -> Retrieve -> Generate
- Simple LangChain Application
- LangChain RAG
- Simple LangGraph Workflow
- LangGraph Tool-Calling Agent
- State Management with LangGraph
- Conditional Routing
- Human Approval Flow
- Multi-Agent Supervisor/Worker Example
- Error + Retry Handling

- Logging / Tracing
- Build One Complete Agentic AI Project

19. Interview + Scenario-Based Preparation

CRITICAL

- Explain LLM in Simple Terms
- Explain RAG Architecture
- Explain Tool Calling
- Explain Agent Architecture
- Explain ReAct
- Explain State vs Memory
- Explain LangChain vs LangGraph
- Explain Function Calling vs MCP
- Design a RAG System
- Design a Tool-Calling Agent
- Design a Multi-Agent System
- How Would You Prevent Infinite Agent Loops?
- How Would You Handle Tool Failure?
- How Would You Prevent Duplicate Transactions?
- How Would You Secure Agent Tools?
- How Would You Handle Prompt Injection?
- How Would You Reduce Hallucinations?
- How Would You Reduce Token Cost?
- How Would You Reduce Latency?
- When Would You NOT Use an Agent?
- When Would You NOT Use Multi-Agent?
- Production Agent Debugging Scenarios